

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА − Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра инструментального и прикладного программного обеспечения (ИППО)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Архитектура, проектирование и разработка программных средств»

Практическая работа № 3

Шаблоны проектирования в архитектуре программных систем

Студенты группы ИКМО-03-25 Фомичев Р.А.

(подпись)

Старший преподаватель *Синицын А.В.*

(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

Шаблоны проектирования в архитектуре ИС «Учёт и обслуживание компьютерной техники»

1. Цель работы и выбор системы

Цель работы — научиться видеть архитектурные проблемы в проектируемой системе и подбирать для них подходящие шаблоны проектирования, аргументируя выбор. Шаблон проектирования — типовое решение часто возникающей задачи, проверенное на практике; это не готовый код и не библиотека, а схема ролей и связей, которую реализуют на любом языке [1].

Объект проектирования продолжает практические работы № 1 и № 2 — информационная система «Учёт и обслуживание компьютерной техники» (далее — система). ИТ-отдел из шести человек ведёт в ней реестр около 2500 единиц техники, выполняет заявки, планирует ТО, проводит инвентаризацию и готовит акты для бухгалтерии. В работе № 2 методом ADD выбрана архитектура верхнего уровня — модульный монолит с SPA-клиентом, фоновым обработчиком и очередью в PostgreSQL. Это решения макроуровня; настоящая работа спускается на микроуровень — к классам внутри сервера приложений **Э3** — и уточняет задачу **Т02** плана первой работы (архитектура, модель данных, API). Ссылки F01–F14 указывают на требования первой работы, Э1–Э7 и Д1–Д6 — на элементы и драйверы второй.

Система состоит из четырёх независимых прикладных модулей и двух общих подсистем (таблица 1). Модули сдаются заказчику по очереди, по итерациям первой работы, поэтому у каждого должна быть своя граница и узкий интерфейс. Серверная часть пишется на Python (ПР-1, п. 3.6); на схемах классы названы по-английски, как в коде.

Таблица 1 – Подсистемы системы

№	Подсистема	Обязанности	Требования	Кому нужна
Э3.3	Учёт техники	Реестр и карточка единицы, комплект рабочего места, перемещения с неизменяемой историей, история ремонтов	F01–F04	API-слою, модулям заявок, инвентаризации, отчётов
Э3.4	Заявки и обслуживание	Заявки и статусы, сроки в рабочем времени, журнал работ, ЗИП, график ТО	F05–F09	API-слою, модулю инвентаризации (списание)
Э3.5	Инвентаризация и списание	Ведомости, отметки по штрихкоду, расхождения, процесс списания	F10, F11	API-слою
Э3.6	Отчёты и обмен	Акты, этикетки, отчёты XLSX и PDF, обмен CSV с 1С	F12, F14	всем модулям
Э3.2	Безопасность (общая)	Проверка прав authorize, журнал аудита audit	F13	всем модулям
Э4	Фоновый обработчик	Письма из outbox, проверка просрочки, синхронизация с AD	F05, F07, F09	модулям заявок и учёта

2. Архитектурные проблемы

Проблемы выявлены по трём источникам: связям между модулями на рисунке 1 второй работы, функциональным требованиям F01–F14 и открытым вопросам первой работы (п. 4.1) — формам актов, формату обмена, нормативным срокам и периодичности ТО. Открытые вопросы важны: всё, что от них зависит, будет меняться в ходе итераций, и такие места проекта должны допускать изменения без переделки соседних модулей. Выбраны четыре проблемы (таблица 2).

Таблица 2 – Проблемы проектирования

ID	Проблема	Где возникает	Требования, драйверы	Что будет, если не решить
П1	Как скрыть сложность подсистемы и не дать модулям зависеть от чужих внутренних классов	Учёт техники используют API-слой и три других модуля	F01–F04, F11, F13; Д2, Д6	Обход authorize и audit, каскад правок при изменении модели, срыв поэтапной сдачи модулей
П2	Как уведомлять несколько компонентов об изменении данных	Смена статуса заявки, просрочка, остаток ЗИП, перемещение	F03, F05, F07, F08, F14; Д3, Д4	Модуль заявок знает всех получателей; каждая новая реакция — правка самого рискованного модуля

ID	Проблема	Где возникает	Требования, драйверы	Что будет, если не решить
П3	Как поддерживать разные алгоритмы одной задачи — расчёта сроков заявок	Аварийные заявки и заявки «Плановое ТО»	F06, F07, F09; Д4	Разрастание условных операторов в трёх местах: назначение срока, пауза, проверка просрочки
П4	Как централизовать создание документов и добавлять новые виды без правки клиентов	Акты, этикетки, ведомости, отчёты, CSV для 1С	F03, F10–F12, F14; О4	Код форм размазан по трём модулям; смена формы акта затрагивает каждый

2.1. П1. Сложность подсистемы и связи между модулями

Модуль «Учёт техники» — центральный: внутри него шесть классов (хранилища единиц, истории перемещений и истории ремонтов, служба комплектов, проверка уникальности инвентарного номера, формирование данных для акта). Им пользуются API-слой, модуль инвентаризации (поиск единицы по штрихкоду, перевод в «Списано»), модуль заявок (нельзя подать заявку на списанную единицу, F11) и модуль отчётов. Если каждый клиент обращается к внутренним классам напрямую, нарушается ограничение второй работы: модуль меняет данные только через свои прикладные сервисы, которые вызывают authorize и audit. Тогда запрос в обход интерфейса может изменить запись без проверки прав и без записи в журнал (сценарий АК2), а любое изменение внутренней модели потребует правок в трёх модулях — и в итерациях, уже сданных заказчику.

2.2. П2. Уведомления при изменении данных

Одно изменение вызывает несколько реакций в разных местах. Смена статуса заявки: письмо заявителю (F05), остановка и возобновление срока при «Ожидает запчасть» (F07), при «Выполнена» — запись затрат в историю ремонтов единицы, без которой списание решается без данных о затратах (ПР-1, п. 1.2). Просрочка — письмо руководителю (F07); остаток ЗИП ниже минимума — письмо (F08); перемещение — акт в очередь выгрузки для бухгалтерии (F03, F14). Источник события не должен знать список получателей: этот список будет расти по ответам заказчика, а модуль заявок по оценке второй работы — элемент

наибольшего риска. Кроме того, ни одно уведомление нельзя потерять (сценарий АК3).

2.3. П3. Разные алгоритмы расчёта сроков

Аварийная заявка получает нормативы реакции и решения по приоритету (1/8, 4/24, 8/40 ч), считает их только в рабочем времени по производственному календарю и останавливает часы в статусе «Ожидает запчасть». Заявка «Плановое ТО» создаётся за 7 дней до срока (F09): её срок — плановая дата по графику, а время реакции не контролируется. Алгоритм нужен в трёх местах — при создании заявки, при смене статуса и при ежеминутной проверке просрочки фоновым обработчиком. Нормы и периодичность ТО — открытые вопросы, и заказчик может добавить новый тип заявок с собственными правилами.

2.4. П4. Централизованное создание документов

Система выпускает шесть видов документов в четырёх форматах: акты перемещения и списания (PDF A4), этикетку со штрихкодом Code 128 (PDF 58 × 40 мм), ведомость расхождений, четыре отчёта в XLSX и PDF (F12) и выгрузку актов в CSV для учётной системы бухгалтерии (F14). Заказывают документы три модуля. Формы актов и формат файла обмена — открытые вопросы первой работы, поэтому именно этот код будет меняться чаще всего. Нужно, чтобы модуль-клиент не зависел от конкретного класса документа и его формата, а новый вид документа добавлялся без правки клиентов.

3. Подбор шаблонов

Для каждой проблемы рассмотрены несколько решений; выбор и причины отказа от альтернатив сведены в таблицу 3 и раскрыты в пп. 3.1–3.4. Критерии выбора одни и те же: связность модулей, трудозатраты одного серверного разработчика (Д6), риск потери данных и уведомлений (Д3), а также стоимость изменений по открытым вопросам.

Таблица 3 – Проблемы, шаблоны и рассмотренные альтернативы

Проблема	Шаблон (группа GoF)	Альтернативы	Почему альтернативы хуже
П1	Фасад (структурный) — по одному на модуль	а) прямой доступ к классам модуля; б) Посредник; в) модули как отдельные службы с REST	а) обход authorize/audit, высокая связность; б) посредник знает все модули и становится «божественным объектом»; в) микросервисы отклонены в ПР-2 из-за Д6
П2	Наблюдатель (поведенческий) + транзакционный outbox из ПР-2	а) прямые вызовы; б) триггеры БД, LISTEN/NOTIFY; в) внешний брокер; г) асинхронная шина в памяти	а) см. раздел 5; б) не знают автора, логика уходит в SQL, NOTIFY теряется без слушателя; в) отклонён в ПР-2; г) теряет события при перезапуске, нарушает АК3
П3	Стратегия (поведенческий)	а) условные операторы по типу заявки; б) Состояние; в) Шаблонный метод через наследование заявок	а) правило разносится по трём методам; б) решает другую задачу — статусы, а не сроки; в) смешивает сущность и правило, усложняет ORM
П4	Фабричный метод (порождающий) с реестром создателей	а) создание через конструктор в каждом модуле; б) Абстрактная фабрика; в) Строитель; г) внешний генератор отчётов	а) дублирование, смена формы затрагивает три модуля; б) нет семейств связанных продуктов; в) документ не собирается клиентом по шагам; г) ещё один процесс и память на сервере 8 ГБ

3.1. Фасад для модулей (П1)

Каждый из четырёх модулей открывает наружу ровно один класс — фасад (**AssetRegistryFacade**, **RequestsFacade**, **InventoryFacade**, **ReportsFacade**). Метод фасада соответствует одному сценарию использования и служит границей транзакции: authorize → действие внутренних классов → audit → commit. Внутренние классы лежат во внутреннем пакете модуля, импорт которого из других модулей запрещает правило проверки зависимостей в автосборке (Т05). Так шаблон закрепляет в коде решение второй работы о единой политике доступа: обойти authorize через другой модуль нельзя, потому что других входов нет.

Посредник рассматривался как способ упорядочить взаимодействие модулей, но здесь нет многосторонних связей, которые он упрощает: зависимости идут в одну сторону — от инвентаризации и заявок к учёту техники и к отчётам. Общий посредник знал бы обо всех модулях и рос бы с каждой итерацией. Вынос модулей в отдельные службы с REST-интерфейсом дал бы ту

же изоляцию, но был отклонён во второй работе: несколько процессов на сервере с 8 ГБ ОЗУ и межсервисные контракты не по силам одному серверному разработчику.

Риск шаблона — «толстый» фасад, в который постепенно переезжает логика. Правило: фасад только координирует, бизнес-правила живут во внутренних классах; ориентир — не более 10–12 методов на фасад, при превышении модуль делится.

3.2. Наблюдатель для уведомлений (П2)

Издатели — фасады модулей, служба ЗИП и задание проверки просрочки — публикуют доменное событие в шину **DomainEventBus** (роль Subject) и не знают, кто его получит. Наблюдатели реализуют интерфейс **EventHandler** с одним методом handle(event); подписки собраны в одном месте — классе **EventWiring**, где их можно прочитать списком. Состав событий и подписчиков приведён в таблице 4.

Таблица 4 – События и наблюдатели

Событие	Издатель	Наблюдатели и их реакция	Требования
RequestStatusChanged	RequestsFacade	RequesterNotifier — письмо заявителю в outbox; SlaClock — пауза и возобновление срока, фиксация времени решения; RepairHistoryRecorder — затраты в историю ремонтов единицы	F05–F07, F11
RequestOverdue	OverdueCheckJob (Э4)	ManagerNotifier — письмо руководителю ИТ-отдела	F07
StockBelowMinimum	SparePartsService	ManagerNotifier — письмо о пополнении ЗИП	F08
AssetMoved	AssetRegistryFacade	MoveActExporter — акт перемещения в очередь выгрузки для 1С	F03, F14

Важная деталь — комбинация с транзакционным outbox из второй работы. Обработчики вызываются синхронно, в транзакции издателя, и сами ничего во внешний мир не отправляют: письмо лишь записывается в таблицу outbox, а уходит позже через фоновый обработчик **Э4**. Поэтому событие не теряется при перезапуске, а ошибка обработчика откатывает и исходное изменение — данные остаются согласованными. Асинхронная шина в памяти выглядела бы проще, но потеряла бы события при сбое процесса и нарушила сценарий АК3. Триггеры БД

и LISTEN/NOTIFY отклонены по причине из второй работы: триггер не знает автора действия, а уведомление NOTIFY пропадает, если в этот момент нет слушателя.

Журнал аудита сознательно не сделан наблюдателем: запись аудита обязательна для каждого изменения (F13), и её отсутствие должно быть видно в коде метода фасада, а не зависеть от того, зарегистрирована ли подписка.

3.3. Стратегия для расчёта сроков (П3)

Интерфейс **SlaPolicy** объявляет три операции — назначить сроки, учесть смену статуса и проверить просрочку. Реализации: **IncidentSlaPolicy** (нормы по приоритету, рабочее время, пауза в «Ожидает запчасть») и **PlannedMaintenanceSlaPolicy** (срок — дата ТО по графику). Контекст **DeadlineService** получает нужную стратегию из реестра по типу заявки. Его вызывают фасад заявок при создании, наблюдатель SlaClock при смене статуса и фоновое задание проверки просрочки, так что правило расчёта находится в одном классе, а не в трёх местах.

Нормы 1/8, 4/24 и 8/40 ч остаются данными: они хранятся в таблице, и руководитель меняет их без изменения кода. Стратегия отвечает за способ расчёта, а не за числа, поэтому для приоритетов отдельные классы не созданы — они отличаются только значениями. Новый тип заявок с другими правилами — это новый класс и одна строка регистрации.

Рассматривался шаблон Состояние для статусной модели заявки (F06). Он подходит, когда поведение объекта сильно зависит от состояния; здесь шесть статусов отличаются лишь допустимыми переходами, и их достаточно описать таблицей переходов. Шесть классов состояний были бы оверинжинирингом — использованием шаблона «ради галочки». Условные операторы по типу заявки сейчас короче стратегии, но правило размазалось бы по трём методам, и каждое уточнение норм по открытым вопросам затрагивало бы все три.

3.4. Фабричный метод для документов (П4)

Абстрактный создатель **DocumentCreator** задаёт общий порядок выпуска документа в методе produce(): собрать данные → проверить → получить отрисовщик через фабричный метод createRenderer() → отрисовать → сохранить. Конкретные создатели (**MoveActCreator**, **WriteOffActCreator**, **AssetLabelCreator**, **RequestsReportCreator**, **AccountingCsvCreator**) сами решают, какой продукт — **PdfA4Renderer**, **PdfLabelRenderer**, **XlsxRenderer** или **CsvRenderer** — создать. Реестр **DocumentFactory** сопоставляет виду и формату документа нужного создателя, а модули обращаются только к фасаду отчётов: makeDocument(вид, источник, формат).

Когда заказчик утвердит формы актов, изменится один класс создателя, а не код трёх модулей; когда будет согласован формат обмена с 1С — один класс CsvRenderer. Абстрактная фабрика не выбрана, потому что у системы нет семейств взаимосвязанных продуктов: каждый документ — один продукт одного формата. Строитель полезен для пошаговой сборки сложного объекта клиентом, а здесь клиент лишь заказывает документ; при необходимости Строитель можно применить внутри XlsxRenderer. Внешний генератор отчётов потребовал бы ещё одного процесса и памяти на сервере, что противоречит драйверу Д6.

Ещё два шаблона упомянуты для полноты. Адаптер уже применён во второй работе (элемент Э6 — LDAP, SMTP, каталог CSV) и здесь не пересматривается. Одиночка сознательно не используется: единственный экземпляр шины и реестра документов создаётся в точке сборки приложения и передаётся через конструкторы, без глобального состояния, — так классы проще тестировать.

4. Архитектурные схемы

Схемы построены в нотации UML. Принадлежность элемента шаблону показана двумя способами: стереотипом с ролью в шаблоне («Facade», «Subject», «Observer», «Strategy», «Creator», «Product» и т. д.) и цветом: синий — Фасад, зелёный — Наблюдатель, оранжевый — Стратегия, фиолетовый — Фабричный

метод; белым показаны классы, не играющие роли в шаблонах. На рисунке 1 — обзор всех четырёх шаблонов в модулях сервера приложений.

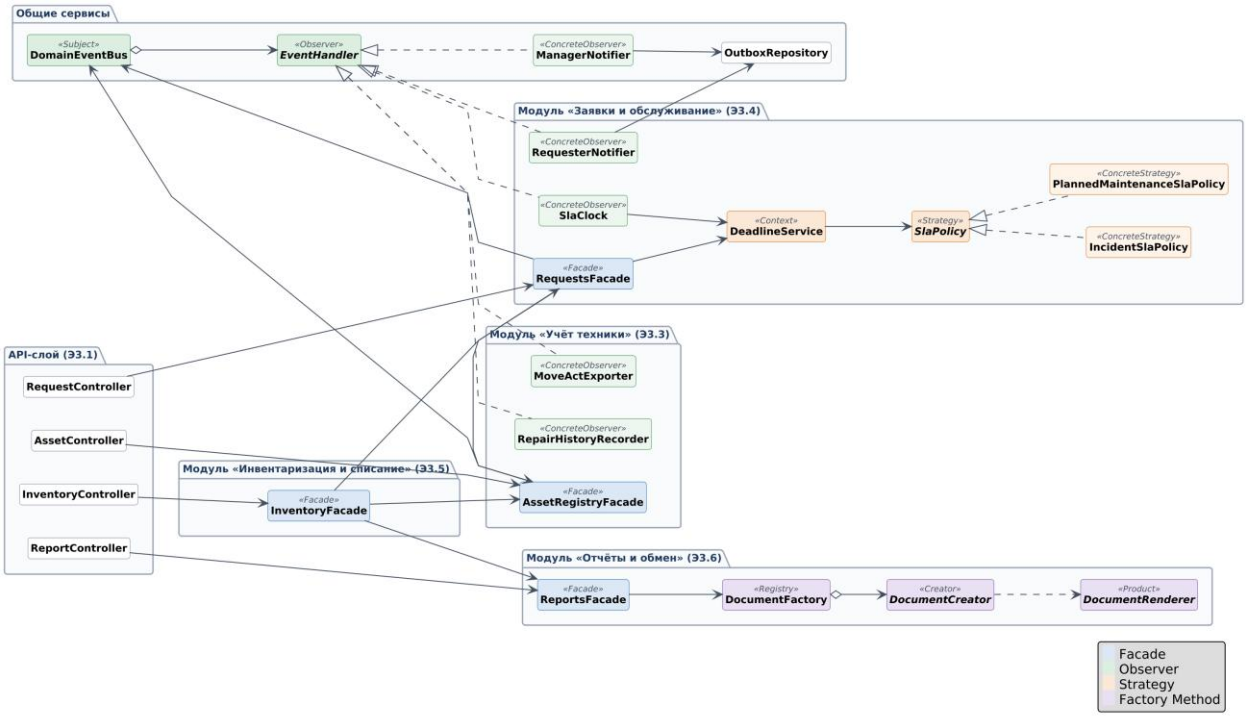


Рисунок 1 – Обзорная диаграмма классов: шаблоны в модулях сервера приложений

Каждый контроллер API-слоя обращается только к фасаду своего модуля, модули связаны между собой тоже только через фасады. Стратегия находится внутри модуля заявок, фабричный метод — внутри модуля отчётов, а шина событий и интерфейс наблюдателя — в общих сервисах, потому что подписчики есть в разных модулях. Детальные диаграммы каждого шаблона приведены на рисунках 2–5.

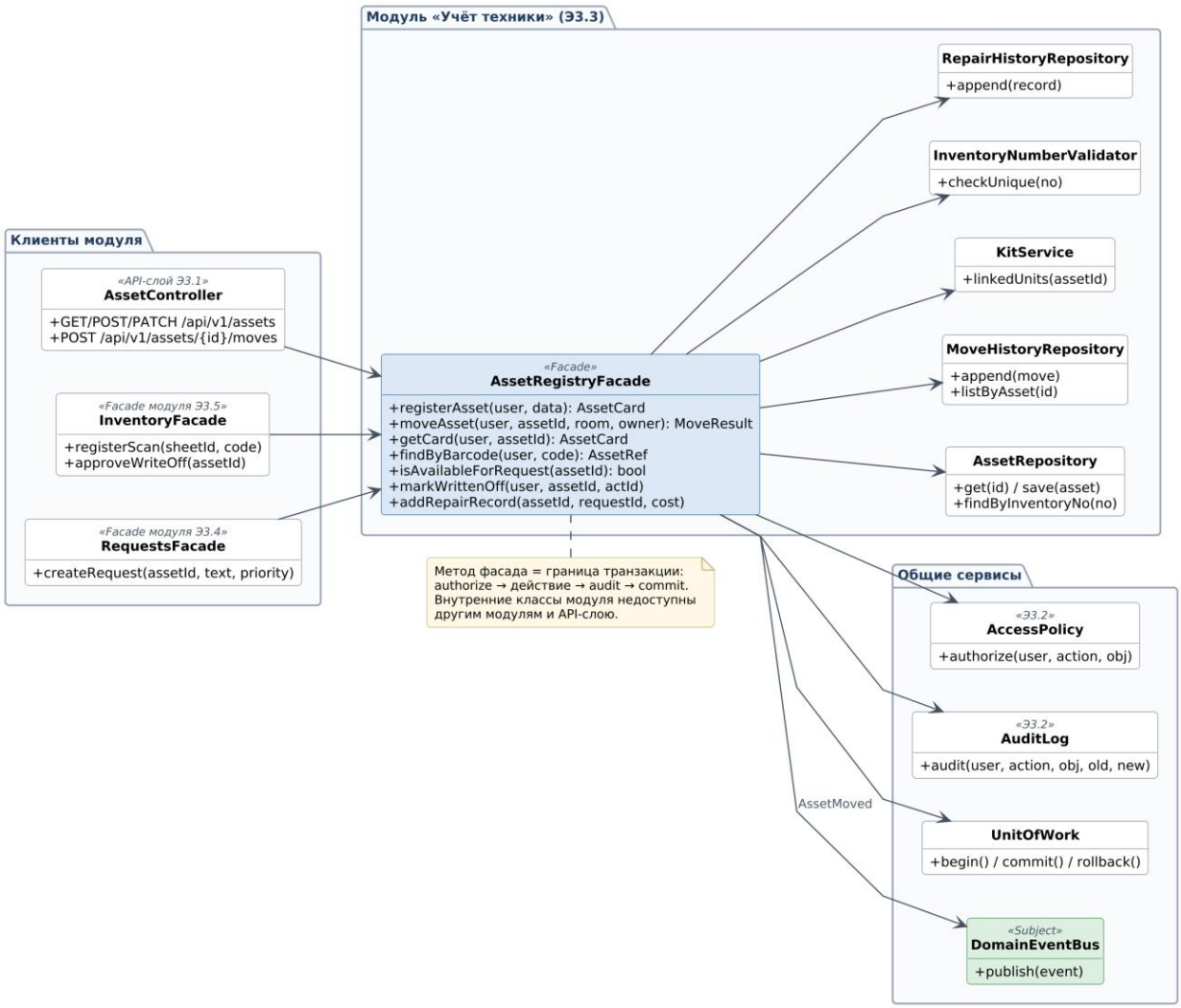


Рисунок 2 – Диаграмма классов шаблона Фасад для модуля «Учёт техники»

Три клиента модуля видят только AssetRegistryFacade. За фасадом — пять внутренних классов модуля и общие сервисы: проверка прав, журнал аудита, управление транзакцией и шина событий, в которую фасад публикует событие AssetMoved.

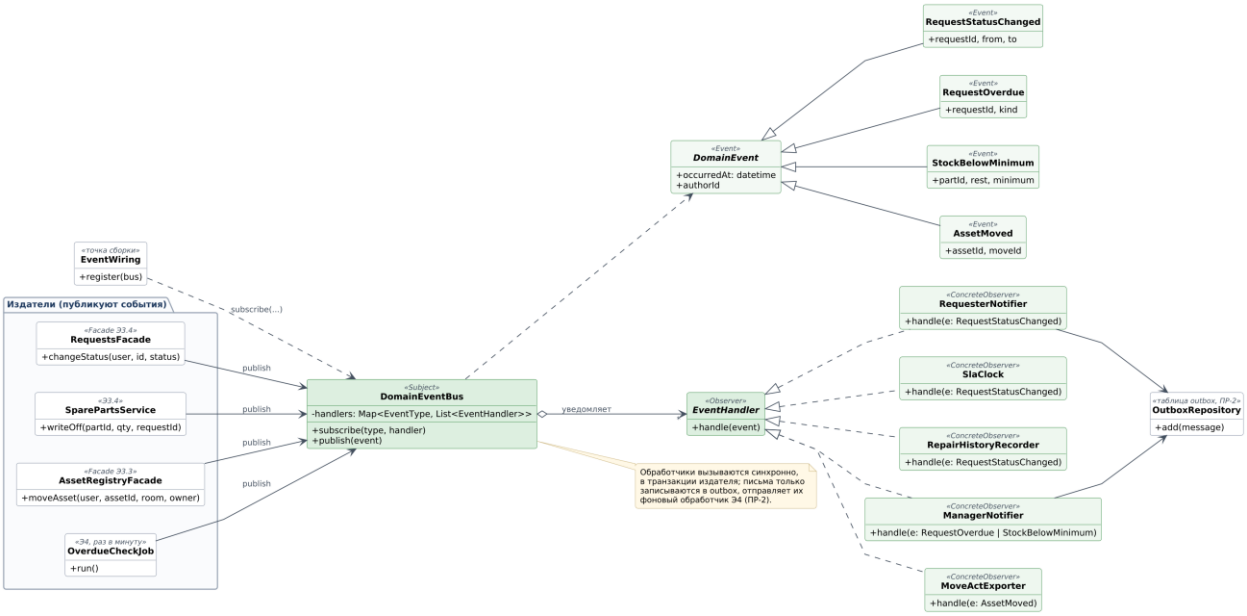


Рисунок 3 – Диаграмма классов шаблона Наблюдатель

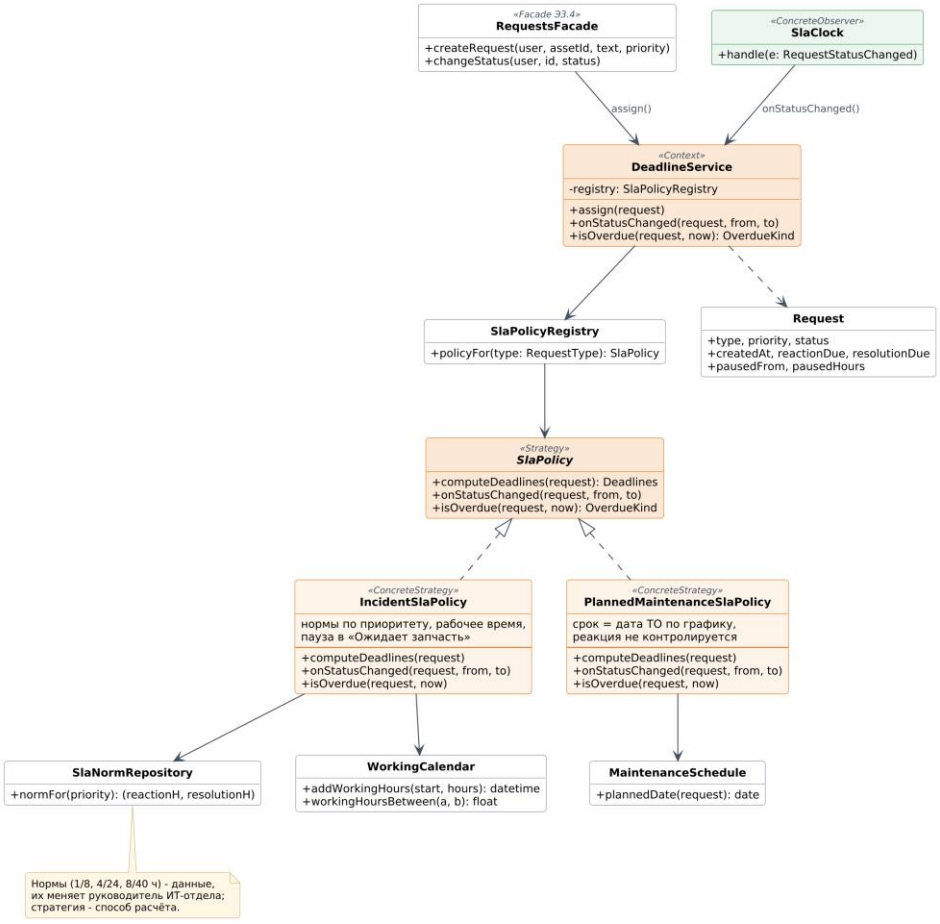


Рисунок 4 – Диаграмма классов шаблона Стратегия для расчёта сроков

SlaClock на рисунке 4 одновременно конкретный наблюдатель и клиент контекста стратегии: так два шаблона соединяются без прямой зависимости фасада заявок от правил расчёта при смене статуса.

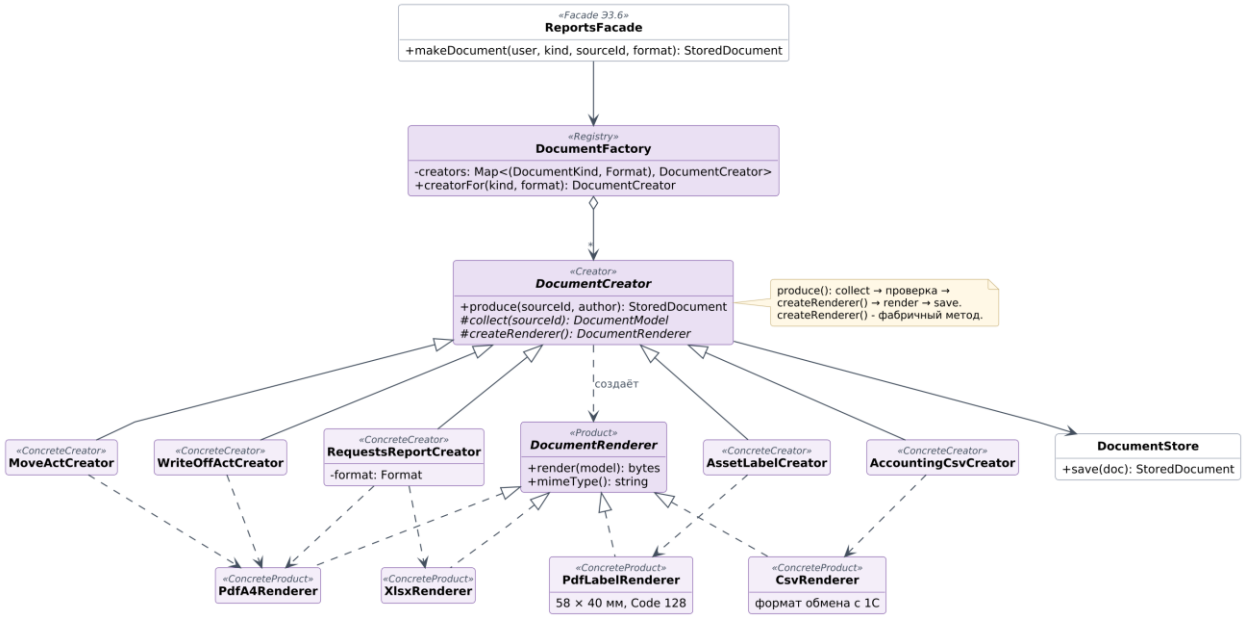


Рисунок 5 – Диаграмма классов шаблона Фабричный метод для документов

Совместную работу шаблонов показывает диаграмма последовательности на рисунке 6 — специалист переводит заявку в статус «Выполнена».

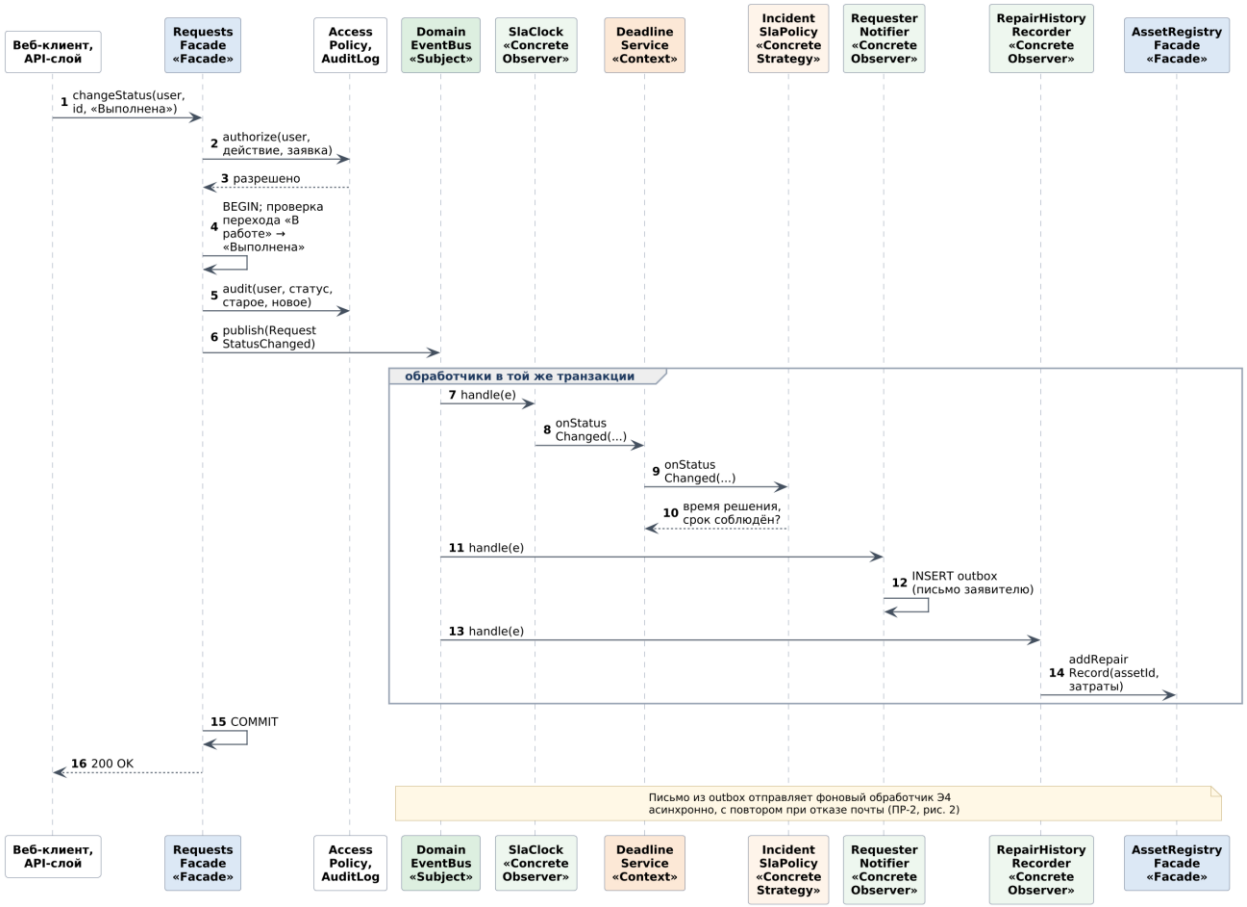


Рисунок 6 – Последовательность смены статуса заявки: Фасад, Наблюдатель и Стратегия

Фасад заявок проверяет права, проверяет допустимость перехода, пишет аудит и публикует событие (сообщения 1–6). В той же транзакции шина вызывает трёх наблюдателей: SlaClock через контекст DeadlineService передаёт расчёт стратегии IncidentSlaPolicy, RequesterNotifier записывает письмо в outbox, RepairHistoryRecorder передаёт затраты в модуль учёта техники — снова через фасад (сообщения 7–14). После фиксации транзакции клиент получает ответ, а письмо отправляет фоновый обработчик. Роли всех классов на схемах сведены в таблицу 5.

Таблица 5 – Классы и их роли в шаблонах

Шаблон	Роль в шаблоне	Классы системы	Модуль
Фасад	Facade	AssetRegistryFacade, RequestsFacade, InventoryFacade, ReportsFacade	Э3.3–Э3.6
Фасад	Подсистема	AssetRepository, MoveHistoryRepository, KitService, InventoryNumberValidator, RepairHistoryRepository	Э3.3
Наблюдатель	Subject	DomainEventBus	общие сервисы
Наблюдатель	Observer / ConcreteObserver	EventHandler / RequesterNotifier, SlaClock, RepairHistoryRecorder, ManagerNotifier, MoveActExporter	Э3.3, Э3.4, общие
Наблюдатель	Событие	RequestStatusChanged, RequestOverdue, StockBelowMinimum, AssetMoved	общие сервисы
Стратегия	Context	DeadlineService (выбор через SlaPolicyRegistry)	Э3.4
Стратегия	Strategy / ConcreteStrategy	SlaPolicy / IncidentSlaPolicy, PlannedMaintenanceSlaPolicy	Э3.4
Фабричный метод	Creator / ConcreteCreator	DocumentCreator / MoveActCreator, WriteOffActCreator, AssetLabelCreator, RequestsReportCreator, AccountingCsvCreator	Э3.6
Фабричный метод	Product / ConcreteProduct	DocumentRenderer / PdfA4Renderer, PdfLabelRenderer, XlsxRenderer, CsvRenderer	Э3.6

5. Сравнение альтернатив

Для сравнения выбран Наблюдатель: у него есть очевидная простая альтернатива — прямой вызов методов получателей из фасада заявок. Фрагменты метода changeStatus для обоих вариантов приведены ниже, связи — на рисунке 7.

а) Прямые вызовы

```
def change_status(self, user, request_id, new_status):
    req, old = self._check_and_apply(user, request_id, new_status)
    self._deadlines.on_status_changed(req, old, new_status)
    self._mail_queue.add(requester_letter(req))
    if new_status == Status.DONE:
        self._assets.add_repair_record(req.asset_id, req.id, req.cost)
    # каждая новая реакция — новая строка и новая зависимость здесь
```

б) Наблюдатель

```
def change_status(self, user, request_id, new_status):
    req, old = self._check_and_apply(user, request_id, new_status)
    self._bus.publish(RequestStatusChanged(req.id, old, new_status, user.id))


# events_wiring.py — все подписки в одном месте
bus.subscribe(RequestStatusChanged, sla_clock)
bus.subscribe(RequestStatusChanged, requester_notifier)
bus.subscribe(RequestStatusChanged, repair_history_recorder)
```

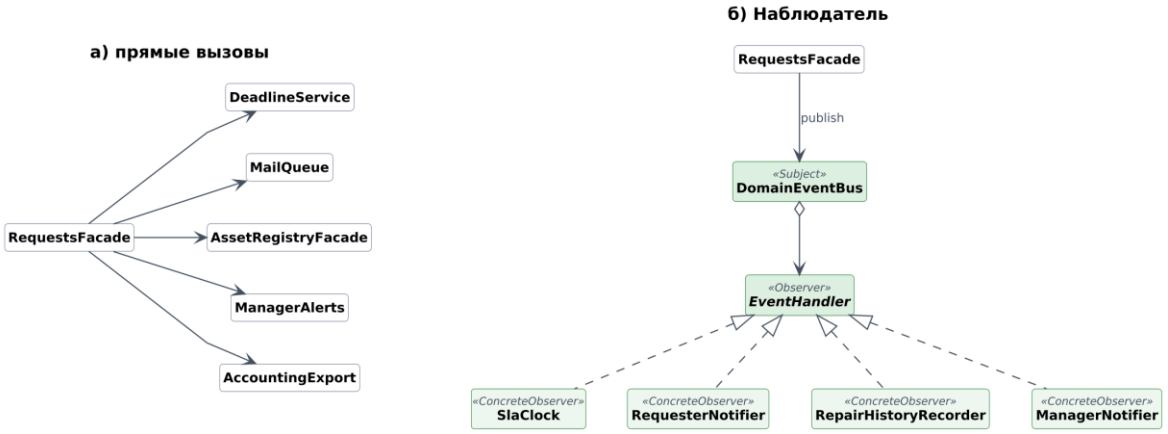


Рисунок 7 – Зависимости фасада заявок: а) прямые вызовы; б) Наблюдатель

Таблица 6 – Сравнение прямых вызовов и шаблона Наблюдатель

Критерий	Прямые вызовы	Наблюдатель
Связность	– RequestsFacade зависит от всех получателей (уже 3–4 класса, в том числе из модуля учёта техники)	+ одна зависимость — шина; получатели не известны издателю
Новая реакция на событие	– правка и повторное тестирование метода самого рискованного модуля	+ новый класс-обработчик и строка в EventWiring; фасад не меняется
Наглядность потока	+ вся последовательность видна в одном методе	– поток распределён; нужно смотреть список подписок
Порядок выполнения	+ задан явно	– по порядку регистрации; на порядок нельзя опираться в логике
Транзакция и ошибки	+ всё в одной транзакции	= то же при синхронной шине; асинхронная шина нарушила бы АК3
Тестирование	– для теста фасада нужны заглушки всех получателей	+ фасад проверяется по опубликованному событию, обработчики — отдельно

Критерий	Прямые вызовы	Наблюдатель
Объём кода и вхождение	+ меньше классов, понятно новичку	– шина, классы событий и обработчиков, правило «где искать реакцию»
Производительность	+ без накладных расходов	= несколько вызовов через словарь подписок — пренебрежимо при десятках событий в минуту

Прямые вызовы выигрывают по простоте и наглядности, пока реакций мало и они не меняются. В системе уже четыре вида событий и пять реакций, а их состав зависит от открытых вопросов первой работы, поэтому решающими стали критерии связности и стоимости изменений: с Наблюдателем добавление реакции не трогает модуль заявок, который вторая работа назвала элементом наибольшего риска. Минусы Наблюдателя компенсируются правилами: все подписки — в одном файле EventWiring, обработчики не зависят от порядка вызова, а автотест проверяет, что у каждого события есть ожидаемые подписчики.

При этом прямой вызов оставлен там, где действие обязательно и должно быть видно в коде: authorize и audit вызываются фасадом явно (п. 3.2). Шаблон применяется к необязательным, растущим реакциям, а не ко всем вызовам подряд.

6. Выводы

Преимущества интеграции шаблонов.

– Фасады закрепили в коде границы модулей из второй работы и единую политику доступа: у каждого модуля один вход, и каждый метод этого входа проверяет права и пишет аудит. Модули можно сдавать по итерациям, не боясь, что изменение внутренней модели сломает уже принятые.
– Наблюдатель вместе с транзакционным outbox отделил источники событий от получателей и сохранил гарантию второй работы: уведомление не теряется при отказе почты и перезапуске сервера.

– Стратегия собрала правила расчёта сроков — самую сложную часть требований (F06–F07) — в один класс на тип заявки; нормы остались данными, которые меняет руководитель без программиста.
– Фабричный метод сосредоточил места, зависящие от открытых вопросов (формы актов, формат обмена с 1С), в отдельных классах: ответы заказчика меняют один класс, а не три модуля.
– Шаблоны дали команде общий словарь: в задачах и обсуждениях достаточно сказать «новый наблюдатель на AssetMoved» или «новый создатель документа».

Новые риски и сложности.

– Больше классов и уровней косвенности: вместо одного метода — фасад, шина, события, обработчики, реестры. Для одного серверного разработчика (Д6) это дополнительная работа в итерации 2; по грубой оценке, около 8–16 ч из 96 ч задачи Т09.
– Поток управления при Наблюдателе неявный: реакцию на событие нужно искать в списке подписок; ошибка в обработчике откатывает исходную операцию пользователя.
– Фасад может разрастись в «божественный объект», а реестры стратегий и создателей — стать местом, которое забывают дополнить при добавлении класса.
– Есть соблазн оверинжиниринга — добавлять шаблоны «на будущее». Поэтому Состояние, Одиночка, Абстрактная фабрика и Посредник рассмотрены и сознательно не применены.

Риски закрываются правилами разработки, которые проверяются автоматически в сборке (Т05): запрет импорта внутренних пакетов чужих модулей, ориентир по числу методов фасада, тест полноты подписок и реестров, а также правило, что обработчик события не опирается на порядок вызова. Следующий шаг — детальное проектирование модуля «Заявки и обслуживание» в итерации 2 по выбранным шаблонам.

7. Ссылки

[1] Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер (оригинал: Gamma E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994).

[2] Fowler M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

[3] Freeman E., Robson E. Head First Design Patterns. Sebastopol: O'Reilly, 2004.

[4] Richardson C. Pattern: Transactional outbox: https://microservices.io/patterns/data/transactional-outbox.html

[5] Фомичев Р.А. Практическая работа № 1. Программный проект. Процесс разработки ПО (документ требований ИС «Учёт и обслуживание компьютерной техники»). РТУ МИРЭА, 2026.

[6] Фомичев Р.А. Практическая работа № 2. Атрибутный метод проектирования архитектуры. РТУ МИРЭА, 2026.

Заключение

Для системы учёта и обслуживания компьютерной техники выявлены четыре проблемы проектирования микроуровня и для каждой выбран шаблон: Фасад скрывает сложность модулей и закрепляет единую политику доступа, Наблюдатель в связке с транзакционным outbox доставляет уведомления об изменениях без потерь, Стратегия разделяет алгоритмы расчёта сроков заявок, Фабричный метод централизует создание документов, формы которых ещё уточняются. Для каждого выбора рассмотрены и отклонены альтернативы; построены обзорная и четыре детальные диаграммы классов и диаграмма последовательности с пометкой ролей шаблонов. Сравнение Наблюдателя с прямыми вызовами показало, что шаблон выигрывает по связности и стоимости изменений ценой меньшей наглядности потока. Интеграция шаблонов

уменьшила связность и локализовала будущие изменения, но добавила классы и косвенность; эти риски ограничены автоматически проверяемыми правилами разработки.